

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4 Total Marks 20 Duration :60 Minutes Annexure A
Mock Interview

Code of Conduct:

I __Harshavardhan .RV (192472163)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harshavardhan .RV

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

1. OpenCV Pipeline for Processing Multiple Images

A complete OpenCV pipeline can be designed as:

Folder → Read Images → Preprocess → Process → Display/Save → Release Resources

Steps

1. **Import libraries**
 - Use cv2 for image processing and os or glob for folder handling.
2. **Get image paths**
 - Collect all .jpg, .png, .jpeg files from the folder.
3. **Read images**
 - Use cv2.imread() to load each image.
4. **Preprocess**
 - Resize images using cv2.resize().
 - Convert color using cv2.cvtColor() if required.
 - Apply filtering such as Gaussian blur when necessary.
5. **Process**
 - Perform operations such as edge detection, thresholding, or object detection.

Display or save

- Use cv2.imshow() to display results.
- Use cv2.imwrite() to save processed images.

6. Release resources

- Use cv2.destroyAllWindows() after processing.

Example Code

```
import cv2
import glob

images = glob.glob("images/*.jpg")

for file in images:
    img = cv2.imread(file)

    if img is None:
        continue

    img = cv2.resize(img, (640, 480))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    cv2.imshow("Image", gray)

    if cv2.waitKey(500) == 27:
        break

cv2.destroyAllWindows()
```

Justification and Challenges

Processing images one at a time reduces memory usage. Checking img is None prevents errors from corrupted or unsupported files. For large folders, resizing images and processing only required files improves speed. Multithreading or batch processing can further improve performance.

2. Debugging a Video That Does Not Load

A structured debugging approach is:

Check File → Check Path → Check Codec → Open Video → Check Frames → Check OpenCV Installation

Steps

1. Verify the file

- Make sure the video exists and can be played using another media player.

2. Check the file path

- Use the correct absolute or relative path.

3. Open using VideoCapture()

```
cap = cv2.VideoCapture("video.mp4")
```

4. Check whether it opened

```
if not cap.isOpened():
```

```
    print("Video cannot be opened")
```

5. Check frame reading

```
ret, frame = cap.read()
```

```
if not ret:
```

```
    print("Frame cannot be read")
```

6. Check codec support

- The video codec may not be supported by the installed OpenCV/FFmpeg configuration.

7. Check permissions

- Ensure the program has permission to access the file.

Possible Causes

- Incorrect file path
- Missing or unsupported codec
- Corrupted video
- Unsupported format
- Incorrect OpenCV installation
- Permission problems
- Empty or invalid video file

Finally, use `cap.release()` to release the video resource.

3. Overlaying Text and Shapes on Live Video

The pipeline is:

Camera → Capture Frame → Draw Text/Shapes → Display → Repeat

Important OpenCV Functions

- `cv2.VideoCapture()` – captures live video.
- `cv2.putText()` – adds text.
- `cv2.rectangle()` – draws rectangles.
- `cv2.circle()` – draws circles.
- `cv2.line()` – draws lines.

- `cv2.imshow()` – displays the frame.
- `cv2.waitKey()` – controls frame timing and keyboard input.

Example Code

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    cv2.putText(frame, "Live Video",
                (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX,
                1, (255, 255, 255), 2)

    cv2.rectangle(frame, (50, 70), (250, 200),
                  (0, 255, 0), 2)

    cv2.circle(frame, (400, 150), 50,
                (0, 0, 255), 2)

    cv2.imshow("Camera", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Efficiency

Drawing directly on the captured frame avoids unnecessary image copies. `waitKey(1)` provides near-real-time display. For better performance, the camera resolution can be reduced and unnecessary processing should be avoided.

4. Processing an Image That Is Too Dark

A dark image can result from **poor lighting, camera exposure, incorrect image processing, or low pixel intensity**.

Processing Pipeline

Read Image → Analyze Brightness → Enhance → Improve Contrast → Display

Methods

1. **Brightness adjustment**
 - Add a constant value to pixel intensities.
2. **Contrast enhancement**
 - Use `cv2.convertScaleAbs()`.
3. **Histogram Equalization**
 - Use `cv2.equalizeHist()` on a grayscale image.
4. **CLAHE**
 - `cv2.createCLAHE()` improves local contrast and is useful when different parts of the image have different brightness levels.

Example

```
import cv2
```

```
img = cv2.imread("dark.jpg")
```

```
if img is not None:
```

```
    bright = cv2.convertScaleAbs(img, alpha=1.5, beta=40)
```

```
    gray = cv2.cvtColor(bright, cv2.COLOR_BGR2GRAY)
```

```
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```
    enhanced = clahe.apply(gray)
```

```
    cv2.imshow("Enhanced Image", enhanced)
```

```
    cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

This Improves Visibility

alpha increases contrast while beta increases brightness. CLAHE enhances local regions without excessively increasing the brightness of already bright areas. This makes details in dark regions easier to see while reducing the risk of overexposure.